

**UNIVERSIDAD CATÓLICA DE SANTA MARÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS**

PRÁCTICA N° 07:**PROYECTO DE FIN DE CARRERA****I****OBJETIVOS**

- Realizar la implementación del software del proyecto.

II**TEMAS A TRATAR**

- Construcción del código
- Construcción de la base de datos
- Control de versiones del código
- Calidad del código
- Desarrollo de componentes
- Metodología CRISP-DM para estructurar la implementación de proyectos con Inteligencia artificial

III**MARCO TEÓRICO****CONSTRUCCION DEL CODIGO**

La construcción del código es el proceso mediante el cual se transforma un diseño en una solución funcional a través de la programación. Este proceso debe seguir una serie de etapas organizadas para asegurar que el código sea eficiente, mantenible, y de alta calidad. A continuación, se muestra las etapas clave en la construcción del código.

a) Preparación Previa a la Codificación

Establecer el entorno adecuado y preparar todos los elementos necesarios antes de escribir el código.

Actividades:

- **Definir el entorno de desarrollo:** Seleccionar y configurar las herramientas de desarrollo (IDE, compiladores, depuradores).
- **Establecer estándares de codificación:** Acordar convenciones de estilo, guías de nomenclatura y prácticas recomendadas.
- **Planificación modular:** Definir los componentes y módulos que se construirán, asignando responsabilidades claras para cada uno.

Resultado:

- Entorno de desarrollo configurado y estándares de codificación definidos.

b) Escritura del Código

Convertir el diseño del software en código funcional, siguiendo los principios de programación y las mejores prácticas.

Actividades:

Desarrollar funciones y módulos: Escribir el código para implementar las funcionalidades requeridas, utilizando las estructuras de datos y algoritmos apropiados.

Uso de bibliotecas y frameworks: Aprovechar bibliotecas y frameworks existentes para optimizar el tiempo de desarrollo.

Control de versiones: Realizar commits regulares utilizando sistemas de control de versiones como Git para mantener un historial de cambios.

Buenas prácticas:

Seguir el principio **DRY (Don't Repeat Yourself)** para evitar la duplicación de código.

Mantener la modularidad dividiendo el código en pequeñas funciones reutilizables.

Resultado:

Código escrito de acuerdo con los requisitos y estándares establecidos.

c) Pruebas durante la Codificación

Garantizar que el código funciona correctamente y cumple con los requisitos establecidos.

Actividades:

Pruebas unitarias: Verificar que cada función o módulo funcione correctamente de manera aislada.

Pruebas de integración: Comprobar que los diferentes módulos interactúan correctamente entre sí.

Depuración (Debugging): Identificar y corregir errores o problemas de comportamiento.

Resultado:

Código probado y validado, listo para revisión o integración.

d) Revisión del Código

Mejorar la calidad del código mediante la revisión por otros desarrolladores.

Actividades:

Revisión por pares: Otros desarrolladores revisan el código para garantizar su legibilidad, eficiencia y cumplimiento con los estándares.

Refactorización: Mejorar la estructura del código sin cambiar su funcionalidad, optimizando su legibilidad y rendimiento.

Resultado:

Código revisado y optimizado, con posibles mejoras o sugerencias implementadas.

e) Integración Continua

Asegurar que el código nuevo o actualizado se integre correctamente con el código existente.

Actividades:

Integración continua (CI): Utilizar herramientas como Jenkins o Travis CI para automatizar la integración y ejecutar pruebas con cada cambio.

Pruebas automáticas: Ejecutar automáticamente pruebas unitarias e integradas para asegurar que los cambios no introducen errores en otras partes del sistema.

Resultado:

Código integrado y validado de manera continua, asegurando que no haya conflictos ni fallos críticos.

f) Documentación del Código

Facilitar la comprensión y el mantenimiento del código a largo plazo.

Actividades:

Comentar el código: Incluir comentarios claros que expliquen las partes más complejas del código.

Documentación técnica: Crear documentación adicional que explique el funcionamiento general del sistema, interfaces y flujos principales.

Resultado:

Código documentado y preparado para su mantenimiento y expansión futura.

g) Refactorización y Optimización

Mejorar el rendimiento y la estructura del código sin cambiar su comportamiento.

Actividades:

Refactorización del código: Simplificar el código para hacerlo más modular y fácil de mantener.

Optimización de rendimiento: Revisar y mejorar algoritmos o estructuras de datos para mejorar la eficiencia del código.

Resultado:

Código optimizado y preparado para ser utilizado en producción.

CONSTRUCCION DE LA BASE DE DATOS

La construcción de una base de datos es un proceso que implica varias etapas clave, desde el análisis inicial de los requisitos hasta su implementación final y mantenimiento. Seguir este enfoque sistemático asegura que la base de datos sea eficiente, segura, y escalable para cumplir con los requisitos del sistema.

A continuación, se muestra las principales etapas en la construcción de una base de datos:

a. Análisis de Requisitos

Comprender las necesidades del negocio y definir los datos que deben almacenarse y cómo serán utilizados.

Actividades:

Recolección de requisitos: Identificar el tipo de datos que se deben almacenar (clientes, productos, transacciones, etc.).

Definir restricciones: Considerar limitaciones técnicas o de negocio, como el volumen de datos, tiempos de respuesta, o requisitos de seguridad.

Especificación de requisitos funcionales: Determinar las funcionalidades necesarias, como consultas, informes o integridad de los datos.

Resultado:

Un documento de requisitos que describe los datos y las funcionalidades necesarias para la base de datos.

b. Diseño Conceptual

Crear un modelo conceptual que represente la estructura lógica de la base de datos.

Actividades:

Identificación de entidades: Definir las entidades principales que representan los objetos sobre los que se almacenarán los datos.

Definir atributos: Determinar las propiedades de cada entidad (por ejemplo, nombre, edad, precio).

Modelado de relaciones: Establecer las relaciones entre las entidades (uno a uno, uno a muchos, muchos a muchos).

Diagrama Entidad-Relación (ERD): Crear un **Diagrama Entidad-Relación** que visualice las entidades, atributos y sus relaciones.

Herramientas comunes:

MySQL Workbench, Lucidchart, DBDesigner.

Resultado:

Un **modelo conceptual** en forma de diagrama ERD que representa la estructura lógica de los datos.

c. Diseño Lógico

Convertir el modelo conceptual en un modelo lógico compatible con un sistema de bases de datos relacional (RDBMS).

Actividades:

Normalización: Aplicar las reglas de normalización (1NF, 2NF, 3NF) para reducir la redundancia y garantizar la integridad de los datos.

Definir claves primarias y foráneas: Asignar claves primarias para identificar de forma única cada registro y claves foráneas para relacionar las tablas.

Definir tipos de datos: Especificar los tipos de datos para cada campo (enteros, texto, fechas, etc.).

Definir restricciones de integridad: Definir restricciones como NOT NULL, UNIQUE y CHECK para garantizar la coherencia de los datos.

Resultado:

Un modelo lógico que describe cómo las entidades, atributos y relaciones se implementarán como tablas y columnas en un RDBMS.

d. Diseño Físico

Objetivo:

Optimizar el modelo lógico para su implementación física teniendo en cuenta el rendimiento y el almacenamiento.

Actividades:

Crear el esquema físico: Definir cómo se almacenarán las tablas y sus relaciones en el sistema de archivos.

Definir índices: Crear índices en los campos más utilizados para mejorar la eficiencia de las consultas.

Particionamiento: Dividir grandes tablas en particiones más pequeñas para mejorar el rendimiento.

Optimización de consultas: Analizar las consultas más comunes y optimizar el diseño de las tablas para maximizar la eficiencia.

Resultado:

Un modelo físico optimizado para el almacenamiento y rendimiento en el sistema de bases de datos.

e. Implementación de la Base de Datos

Crear la base de datos y las estructuras definidas en el sistema de gestión de bases de datos (DBMS) seleccionado.

Actividades:

Escritura de scripts SQL: Escribir scripts SQL para crear tablas, índices, claves primarias, y foráneas, entre otros objetos.

Creación de la base de datos: Ejecutar los scripts SQL en el sistema de base de datos para crear el esquema.

Migración de datos: Si es necesario, importar datos desde sistemas anteriores o archivos.

Configuración de permisos: Establecer roles y permisos para garantizar la seguridad y el control de acceso.

Resultado:

La base de datos está creada e implementada, lista para almacenar y gestionar datos.

f. Pruebas de la Base de Datos

Asegurar que la base de datos funcione correctamente y cumpla con los requisitos especificados.

Actividades:

Pruebas de integridad: Verificar que las restricciones y relaciones (claves foráneas) funcionen correctamente.

Pruebas de rendimiento: Realizar pruebas de carga y estrés para evaluar cómo la base de datos maneja grandes volúmenes de datos y múltiples solicitudes.

Pruebas de recuperación: Evaluar la capacidad de respaldo y recuperación de la base de datos en caso de fallos.

Pruebas de seguridad: Asegurarse de que los permisos de acceso funcionan correctamente y que los datos están protegidos frente a accesos no autorizados.

Resultado:

Una base de datos probada y validada, lista para entrar en producción.

g. Despliegue y Mantenimiento

Poner la base de datos en producción y mantener su operación continua.

Actividades:

Despliegue en producción: Implementar la base de datos en el entorno de producción.

Monitoreo: Implementar herramientas de monitoreo para supervisar el rendimiento y detectar problemas.

Mantenimiento: Realizar ajustes continuos en la base de datos, como agregar índices, particionar tablas o realizar optimizaciones.

Respaldo y recuperación: Implementar sistemas de respaldo automático y planes de recuperación ante fallos.

Resultado:

La base de datos está en producción, con mantenimiento y monitoreo continuo para asegurar su correcto funcionamiento.

CONTROL DE VERSIONES DE SOFTWARE

¿Qué es el control de versiones de software?

El control de versiones, también conocido como "control de código fuente", es la práctica de rastrear y gestionar los cambios en el código de software. Los sistemas de control de versiones son herramientas de software que ayudan a los equipos de software a gestionar los cambios en el código fuente a lo largo del tiempo. A medida que los entornos de desarrollo se aceleran, los sistemas de control de versiones ayudan a los equipos de software a trabajar de forma más rápida e inteligente. Son especialmente útiles para los equipos de DevOps, ya que les ayudan a reducir el tiempo de desarrollo y a aumentar las implementaciones exitosas.

El software de control de versiones realiza un seguimiento de todas las modificaciones en el código en un tipo especial de base de datos. Si se comete un error, los desarrolladores pueden ir hacia atrás en el tiempo y comparar las versiones anteriores del código para ayudar a resolver el error, al tiempo que se minimizan las interrupciones para todos los miembros del equipo.

Para casi todos los proyectos de software, el código fuente es como las joyas de la corona, un activo valioso cuyo valor debe protegerse. Para la mayoría de equipos de software, el código fuente es un repositorio del conocimiento de valor incalculable y de la comprensión sobre el dominio del problema que los desarrolladores han recopilado y perfeccionado con un esfuerzo cuidadoso. El control de versiones protege el código fuente tanto de las catástrofes como del deterioro casual de los errores humanos y las consecuencias accidentales.

Etapas del Control de Versiones de Software

El control de versiones de software es un sistema que permite gestionar los cambios en el código fuente, rastrear su evolución y colaborar de manera efectiva entre los miembros del equipo de desarrollo. Facilita el mantenimiento del software, asegura la integridad del proyecto y permite volver a versiones anteriores cuando es necesario.

a. Selección del Sistema de Control de Versiones

Elegir el sistema de control de versiones (**VCS**) adecuado para gestionar el desarrollo del software, basado en las necesidades del proyecto.

Tipos de Sistemas de Control de Versiones:

Centralizado (CVCS): Utiliza un servidor central para almacenar el código, y los desarrolladores acceden a este servidor para realizar sus cambios. Ejemplo: Subversion (**SVN**).

Distribuido (DVCS): Cada desarrollador tiene una copia completa del repositorio en su máquina local, lo que permite trabajar de manera offline y sincronizar los cambios más tarde. Ejemplo: Git, Mercurial.

Herramientas comunes:

Git: El sistema de control de versiones distribuido más utilizado a nivel global, conocido por su flexibilidad y escalabilidad.

Subversion (SVN): Un sistema de control de versiones centralizado utilizado en algunos proyectos que requieren una estructura más simple.

Resultado:

Sistema de control de versiones seleccionado e instalado para gestionar el proyecto de software.

b. Configuración del Repositorio

Inicializar y configurar el repositorio donde se almacenará el código fuente del software y su historial de versiones.

Actividades:

Inicializar el repositorio: Crear el repositorio que contendrá el código fuente. Si es un sistema distribuido como Git, los desarrolladores tienen una copia local del repositorio.

Configurar archivos ignorados: Definir un archivo .gitignore o su equivalente para excluir archivos innecesarios o específicos del entorno de desarrollo (archivos binarios, temporales, de configuración local).

Subir el repositorio a una plataforma remota: Para facilitar la colaboración, el repositorio se puede alojar en plataformas como GitHub, GitLab o Bitbucket.

Resultado:

Repositorio inicializado y preparado con una estructura básica y archivos correctamente configurados.

c. Ramas y Gestión del Desarrollo

Organizar el trabajo en ramas separadas para permitir el desarrollo paralelo de funcionalidades y la resolución de errores sin afectar la rama principal.

Actividades:

Crear ramas: Dividir el desarrollo en **ramas** que aíslen diferentes aspectos del proyecto. Ejemplo: ramas para nuevas funcionalidades (feature/), correcciones de errores (hotfix/), y mejoras (refactor/).

Trabajar en ramas: Cambiar de una rama a otra para realizar cambios en áreas específicas sin afectar el código principal.

Política de ramas: Adoptar una estrategia de ramas clara, como Git Flow o GitHub Flow, para organizar el trabajo en ramas principales (main o master), de desarrollo (develop) y de características (feature).

Resultado:

Desarrollo organizado en diferentes ramas para facilitar el trabajo colaborativo y evitar conflictos en el código.

d. Control de Cambios: Commit, Push y Pull

Guardar y compartir los cambios de código de manera estructurada, permitiendo que otros miembros del equipo puedan acceder y colaborar en tiempo real.

Actividades:

Commit: Guardar los cambios en el repositorio local con un mensaje descriptivo que explique qué modificaciones se realizaron.

Push: Subir los cambios locales al repositorio remoto para que otros desarrolladores puedan acceder a ellos.

Pull: Obtener los cambios más recientes del repositorio remoto para mantener el repositorio local actualizado.

Buenas prácticas:

Realizar commits pequeños y frecuentes para facilitar el seguimiento de cambios.

Escribir mensajes de commit claros y descriptivos que expliquen los cambios realizados.

Resultado:

Cambios registrados de manera eficiente y sincronizados con el repositorio remoto.

5. Revisión de Código y Fusión de Ramas (Merge)

Asegurar la calidad del código mediante revisiones de código y fusionar los cambios en la rama principal o de desarrollo sin conflictos.

Actividades:

Revisión de código (Code Review): Antes de fusionar los cambios, otros desarrolladores deben revisar el código para asegurar su calidad, eficiencia y cumplimiento de los estándares. Esto puede hacerse utilizando Pull Requests o Merge Requests en plataformas como GitHub o GitLab.

Fusión de ramas (Merge): Una vez que el código ha sido revisado, se puede fusionar la rama de trabajo con la rama principal (main) o de desarrollo (develop).

Resolución de conflictos: Si hay conflictos entre las ramas durante la fusión, estos deben resolverse manualmente antes de completar la fusión.

Resultado:

Código revisado y fusionado con la rama principal, listo para la integración en el proyecto.

e. Versionado y Etiquetado (Tagging)

Controlar las versiones del software mediante un sistema de **versionado** y etiquetar versiones importantes para poder hacer un seguimiento adecuado de las mismas.

Actividades:

Etiquetar versiones: Marcar versiones importantes del código utilizando tags. Estos suelen utilizarse en lanzamientos (releases) o hitos importantes en el proyecto.

- **Ejemplo (en Git):** git tag v1.0.0 para etiquetar una versión v1.0.0.

Versionado semántico: Seguir el versionado semántico (por ejemplo, MAJOR.MINOR.PATCH), donde:

- **MAJOR:** Cambios que rompen la compatibilidad.
- **MINOR:** Nuevas funcionalidades, compatibles con versiones anteriores.
- **PATCH:** Correcciones de errores o mejoras menores.

Resultado:

Versiones del software etiquetadas y organizadas de manera semántica para facilitar la gestión de versiones y lanzamientos.

f. Integración Continua y Entrega Continua (CI/CD)

Automatizar el proceso de integración continua (CI) y entrega continua (CD) para asegurar que los cambios en el código se prueben y desplieguen de manera eficiente.

Actividades:

Integración Continua (CI): Configurar herramientas de CI, como Jenkins, Travis CI o GitLab CI, para automatizar la ejecución de pruebas y verificaciones cada vez que se suben cambios al repositorio.

- Los desarrolladores reciben notificaciones automáticas si las pruebas fallan o si hay errores.

Entrega Continua (CD): Automatizar el proceso de despliegue en entornos de prueba, preproducción o producción una vez que los cambios han sido aprobados.

- **Ejemplo:** Desplegar una nueva versión del software en un servidor de producción automáticamente después de que pase las pruebas.

Resultado:

Flujo automatizado de integración, pruebas y despliegue, lo que acelera el ciclo de desarrollo y asegura que el software esté siempre en un estado funcional y producible.

g. Mantenimiento y Gestión de Incidentes

Mantener la estabilidad del software gestionando cambios, resolviendo incidentes y actualizando el código cuando sea necesario.

Actividades:

Resolución de errores (bug fixes): Crear ramas específicas para correcciones urgentes (por ejemplo, hotfix/) y fusionarlas rápidamente con la rama principal.

Mantenimiento del código: Refactorizar el código regularmente para mejorar su legibilidad, modularidad y eficiencia.

Gestión de incidentes: Si ocurre un error en producción, volver a una versión anterior del software usando el historial de control de versiones.

Resultado:

Software mantenido con un historial detallado de cambios, lo que facilita el rastreo y solución de problemas.

CALIDAD DEL CODIGO

La calidad del código hace referencia a qué tan bien escrito y estructurado está el código fuente, lo que afecta su capacidad para cumplir con los requisitos funcionales y no funcionales. Un código de alta calidad no solo cumple con los objetivos del sistema, sino que también es fácil de leer, fácil de mantener, y robusto frente a errores y cambios.

Factores clave de la calidad del código:

a. Legibilidad:

- El código debe ser claro y comprensible tanto para el autor como para otros desarrolladores.
- **Buenas prácticas:**
 - Utilizar nombres de variables, funciones y clases descriptivos que expliquen su propósito.
 - Comentar el código en áreas complejas para facilitar su comprensión sin sobrecargarlo de comentarios.
 - Seguir una convención de estilo como PEP 8 (Python), Google Style (Java) o Airbnb (JavaScript).

b. Mantenibilidad:

- El código debe estar estructurado de manera que sea fácil de modificar y actualizar en el futuro.
- **Buenas prácticas:**
 - **Modularidad:** Dividir el código en componentes y funciones más pequeñas que tengan una responsabilidad única.
 - **Evitar duplicación** de código, siguiendo el principio DRY (Don't Repeat Yourself).
 - **Refactorización periódica:** Revisar y mejorar el código de forma continua para optimizar su estructura sin cambiar su comportamiento.

c. Eficiencia:

- El código debe ser eficiente en términos de rendimiento y uso de recursos (memoria, CPU).
- **Buenas prácticas:**
 - Seleccionar algoritmos y estructuras de datos apropiados para optimizar el tiempo de ejecución y el uso de memoria.
 - Evitar cálculos redundantes o bucles innecesarios.
 - Utilizar profilers para detectar cuellos de botella y optimizar el rendimiento.

d. Seguridad:

- El código debe estar libre de vulnerabilidades que podrían ser explotadas por atacantes.

- **Buenas prácticas:**
 - Validar y sanear toda la entrada del usuario para evitar ataques como SQL Injection o Cross-Site Scripting (XSS).
 - Utilizar herramientas de análisis estático para identificar vulnerabilidades de seguridad en el código.
 - Seguir principios de seguridad por diseño, como el principio de menor privilegio y la separación de responsabilidades.
- e. **Pruebas y validación:**
 - El código debe estar respaldado por un conjunto completo de pruebas que aseguren que las funcionalidades se comportan correctamente.
 - **Buenas prácticas:**
 - Implementar pruebas unitarias, pruebas de integración y pruebas de sistema para verificar la funcionalidad y la interacción entre componentes.
 - Utilizar herramientas de integración continua (CI) como Jenkins o Travis CI para ejecutar las pruebas automáticamente con cada cambio de código.
 - Seguir la metodología TDD (Test-Driven Development), donde las pruebas se escriben antes de implementar el código.

DESARROLLO DE COMPONENTES

El desarrollo de componentes se refiere a la creación de partes funcionales independientes y reutilizables que pueden integrarse en un sistema más amplio. Este enfoque permite que las aplicaciones sean modulares, escalables, y más fáciles de mantener, ya que los componentes pueden ser desarrollados, probados y actualizados de manera independiente.

Características clave del desarrollo de componentes:

- a. **Modularidad:**
 - Un componente debe tener una responsabilidad clara y realizar una tarea específica dentro del sistema. Esto permite que el sistema sea más flexible, ya que los componentes pueden ser reemplazados o actualizados sin afectar a otros.
 - **Ejemplo:** En una aplicación web, podrías tener componentes como el "módulo de autenticación", "módulo de gestión de usuarios", y "módulo de gestión de productos", cada uno con una responsabilidad clara.
- b. **Reutilización:**
 - Los componentes deben ser diseñados para ser reutilizables en diferentes partes del sistema o incluso en otros proyectos. La reutilización ahorra tiempo y reduce el riesgo de errores.
 - **Ejemplo:** Un componente de inicio de sesión puede ser utilizado tanto en la página principal como en la sección de administración de la aplicación.
- c. **Desacoplamiento:**

- Los componentes deben estar desacoplados tanto como sea posible, es decir, su funcionamiento debe depender mínimamente de otros componentes. Esto facilita su prueba, mantenimiento, y reemplazo.
- **Ejemplo:** Un componente de visualización de datos no debería depender directamente de un componente de acceso a la base de datos, sino que debería recibir los datos a través de una interfaz o servicio.
- d. **Interoperabilidad:**
 - Los componentes deben ser capaces de comunicarse entre sí de manera efectiva, utilizando interfaces claras y definidas.
 - **Ejemplo:** Utilizar API o interfaces estándar para que un componente de frontend se comunice con un componente backend sin conocer los detalles internos de su implementación.
- e. **Escalabilidad:**
 - Los componentes deben estar diseñados de manera que puedan ser escalados según las necesidades del sistema. Un componente bien diseñado puede ejecutarse en diferentes entornos o manejar mayores volúmenes de datos sin problemas.
 - **Ejemplo:** Un componente de procesamiento de imágenes debe ser capaz de manejar el procesamiento en paralelo cuando el número de usuarios o imágenes crece exponencialmente.

METODOLOGÍA CRISP-DM PARA ESTRUCTURAR LA IMPLEMENTACION DE PROYECTOS CON INTELIGENCIA ARTIFICIAL

1. **Comprensión del Negocio (Business Understanding)**
 - **Objetivo:** Entender los objetivos y requisitos del negocio para definir claramente el problema que se va a resolver con los datos.
 - **Actividades:**
 - Determinar los **objetivos del proyecto** desde la perspectiva del negocio.
 - Identificar los **recursos** y las **restricciones**.
 - Formular las **preguntas clave** que deben responderse con los datos.
 - **Ejemplo:** Una empresa de retail podría querer aumentar la lealtad del cliente mediante un modelo predictivo que identifique clientes propensos a abandonar la tienda.
2. **Comprensión de los Datos (Data Understanding)**
 - **Objetivo:** Recopilar y analizar los datos para familiarizarse con su estructura, calidad y valor potencial.
 - **Actividades:**
 - Recopilar los datos iniciales.
 - Realizar un análisis exploratorio para **detectar patrones, valores atípicos** y posibles **inconsistencias**.
 - **Ejemplo:** Analizar datos de transacciones y comportamiento de clientes para ver patrones de compra, frecuencia y valores atípicos.
3. **Preparación de los Datos (Data Preparation)**

- **Objetivo:** Limpiar y transformar los datos en un formato adecuado para los modelos.
 - **Actividades:**
 - **Limpiar** los datos eliminando valores nulos, duplicados o incorrectos.
 - **Transformar** variables para que los datos puedan ser interpretados por los algoritmos (por ejemplo, codificación de datos categóricos).
 - **Generar** variables derivadas que puedan mejorar la capacidad predictiva.
 - **Ejemplo:** Estandarizar valores de ingresos, codificar variables categóricas y realizar la división entre conjuntos de entrenamiento y prueba.
4. **Modelado (Modeling)**
- **Objetivo:** Aplicar algoritmos de minería de datos o machine learning para crear modelos predictivos.
 - **Actividades:**
 - Seleccionar y aplicar **algoritmos** como árboles de decisión, regresión, redes neuronales, etc.
 - **Ajustar hiperparámetros** para optimizar el rendimiento del modelo.
 - **Validar** los modelos con los conjuntos de prueba.
 - **Ejemplo:** Crear un modelo de clasificación para predecir qué clientes probablemente abandonarán la tienda.
5. **Evaluación (Evaluation)**
- **Objetivo:** Evaluar el rendimiento del modelo en base a las métricas definidas en la fase de comprensión del negocio.
 - **Actividades:**
 - Revisar si los resultados cumplen los **objetivos del negocio**.
 - Validar que el modelo sea **robusto** y útil para la toma de decisiones.
 - Decidir si el modelo está listo para el despliegue o necesita mejoras.
 - **Ejemplo:** Evaluar el modelo de clasificación basado en precisión, sensibilidad, y tasa de error para asegurarse de que responde a los objetivos comerciales.
6. **Despliegue (Deployment)**
- **Objetivo:** Implementar el modelo en producción para que pueda generar valor para el negocio.
 - **Actividades:**
 - Desarrollar una **estrategia de despliegue** (puede ser una API o un software que use el modelo).
 - Monitorear el **rendimiento del modelo** en el entorno de producción y realizar ajustes si es necesario.
 - Entrenar al personal en el uso del modelo y los resultados.
 - **Ejemplo:** Desplegar el modelo en un sistema de recomendación en línea que sugiera productos basados en el comportamiento de compra de los clientes.

Ciclo Iterativo

- CRISP-DM no es un proceso lineal. Las fases son iterativas, lo que significa que a medida que se avanza en una fase, es posible que se necesite volver a una fase anterior para ajustar aspectos clave (como la comprensión de los datos o el ajuste de los modelos).

Ventajas de CRISP-DM

- **Flexibilidad:** Se adapta a diferentes industrias y tipos de proyectos.
- **Enfoque claro:** Cada fase está orientada hacia el cumplimiento de los objetivos del negocio, lo que garantiza que el proyecto se mantenga alineado con las necesidades organizacionales.
- **Iteración:** Facilita mejoras continuas en cada fase, permitiendo volver atrás cuando sea necesario para refinar el proceso.

IV

ACTIVIDADES

1. Escenario General

- Contexto: La Universidad Católica de Santa María con una fuerte orientación hacia la innovación tecnológica y la resolución de problemas reales.
- Objetivo: Desarrollar la implementación del software del proyecto.
- Enfoque: Fomentar la investigación aplicada y la colaboración interdisciplinaria.

2. Experiencia de Práctica N° 07

Desarrollar:

Para el desarrollo de esta guía Ud. debe de seleccionar el CASO A o el CASO B de acuerdo al producto que se desea implementar.

CASO A: implementación de una aplicación móvil, aplicación web, sistema de información, etc.

- Construcción del código
- Construcción de la base de datos
- Control de versiones del código
- Calidad del código
- Desarrollo de componentes

CASO B: Implementación de proyecto que considere el entrenamiento de modelos de inteligencia artificial (machine learning, red neuronal, etc.), utilizar la metodología CRISP-DM

V

REFERENCIAS

1. Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
2. McConnell, S. (2004). *Code Complete* (2nd ed.). Microsoft Press.
3. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.
4. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th ed.). McGraw-Hill Education.
5. Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress.
6. Loeliger, J., & McCullough, M. (2012). *Version Control with Git: Powerful Tools and Techniques for Collaborative Software Development* (2nd ed.). O'Reilly Media.
7. Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
8. Hunt, A., & Thomas, D. (1999). *The Pragmatic Programmer: Your Journey to Mastery*. Addison-Wesley.
9. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
10. Szyperski, C. (2002). *Component Software: Beyond Object-Oriented Programming* (2nd ed.). Addison-Wesley.
11. Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS Inc.

PROYECTOS DE FIN DE CARRERA
RÚBRICA
PRÁCTICA N° 07

ITEMS	EXCELENTE 4 PUNTOS	BUENO 3 PUNTOS	REGULAR 2 PUNTOS	DEFICIENTE 1 PUNTO	INEXISTENTE 0 PUNTO
Informe 10 POR CIENTO	El informe está bien estructurado, es claro y detallado, con una presentación profesional. Incluye todos los elementos requeridos y referencias adecuadas.	El informe está bien estructurado y es claro, pero puede faltar algún detalle menor o tener errores menores. Incluye la mayoría de los elementos requeridos y referencias adecuadas.	El informe es entendible, pero presenta varios errores en la estructura o contenido. Falta alguno de los elementos requeridos o referencias.	El informe tiene problemas significativos en la estructura o claridad. Falta la mayoría de los elementos requeridos o las referencias son inadecuadas.	El informe está incompleto o no se presenta, y no incluye los elementos requeridos ni referencias.
Exposición 20 POR CIENTO	La exposición es clara, bien organizada y profesional. La información se presenta de manera efectiva y se responde a todas las preguntas con profundidad.	La exposición es clara y bien organizada, pero puede tener algunas áreas menores de mejora. Responde a la mayoría de las preguntas adecuadamente.	La exposición es entendible, pero tiene problemas en la organización o en la presentación. Responde parcialmente a las preguntas.	La exposición es confusa o mal organizada, con falta de claridad en la presentación. Responde de manera deficiente a las preguntas.	La exposición es incompleta, desorganizada o no se realiza. No responde a las preguntas o no aborda los temas requeridos.
Comprensión del tema 20 POR CIENTO	Demuestra un profundo conocimiento del tema de construcción del código, componentes y bases de datos.	Comprende los conceptos básicos de construcción del código, componentes y bases de datos.	Muestra un conocimiento superficial de la construcción del código, componentes y base de datos.	Tiene dificultades para comprender los conceptos básicos de la construcción del código, componentes y base de datos.	No comprende el tema.

Aplicación práctica 30 POR CIENTO	Diseña y construye la base de datos, componentes y aplica buenas prácticas de programación en el código y documenta el proceso de forma clara y concisa.	Completa el diseño y la construcción de la base de datos, componentes y aplica buenas prácticas de programación con algunos errores menores.	Tiene dificultades para completar la construcción de la base de datos, componentes y carece de buenas prácticas de programación.	El diseño y la construcción de la base de datos, componentes y/o código son incompletos o incorrectos.	No diseña ni construye la base de datos, componentes y código.
Uso de herramientas 20 POR CIENTO	Utiliza las herramientas de desarrollo de bases de datos, control de versiones y programación de manera efectiva y eficiente.	Utiliza las herramientas de desarrollo de bases de datos, control de versiones y programación de manera aceptable.	Tiene dificultades para utilizar las herramientas de desarrollo de bases de datos, control de versiones y programación.	No es capaz de utilizar las herramientas de desarrollo de bases de datos, control de versiones y programación.	No utiliza las herramientas de desarrollo de bases de datos, control de versiones y programación.